

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : DSA0307 – Natural Language Processing

A. Assignment Information

Particular	Details
Department:	CSE/IT/AI&DS/AI&ML
Programme:	CSE
Course Code & Course Name	DSA0307 – Natural Language Processing
Academic Year / Batch	2026
Faculty Name	Dr. K. Anbazhagan
Assignment Title	Design and implement a Python-based NLP application for real-time text processing using tokenization, stemming/lemmatization, POS tagging, and sentiment or text classification
Date of Issue	31/08/2026
Date of Submission	02/09/26
Maximum Marks	100
Course Outcome(s) – CO	CO2 - To design an innovative application using NLP components
Bloom's Taxonomy Level	L3 – Apply, L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG8 - Decent Work and Economic Growth & SDG9 - Industry, Innovation and Infrastructure
Industry / Societal Relevance	

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Problem Understanding and Formulation
6. Requirements and Constraints
7. NLP Concepts Used
8. Proposed System
9. System Architecture
10. Algorithm and Pseudocode
11. Dataset Description
12. Implementation
13. Preprocessing Results
14. Sentiment Classification
15. Performance Evaluation
16. Preprocessing Comparison
17. Results and Screenshots
18. Analysis and Engineering Decision
19. Limitations
20. Broader Considerations
21. Conclusion
22. Student Reflection
23. Future Enhancements
24. References
25. Final Submission Checklist

1.ABSTRACT

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to process and analyze human language. This project presents a Python-based NLP application for real-time text processing and sentiment classification. The application accepts a sentence from the user and processes it through multiple NLP stages, including tokenization, stop-word removal, stemming, lemmatization, and Part-of-Speech (POS) tagging.

After preprocessing, the text is converted into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). A Logistic Regression machine-learning classifier is then used to classify the input text into Positive or Negative sentiment. The system also evaluates the classification model using accuracy, precision, recall, and F1-score.

An additional comparison is performed between classification using raw text and classification using preprocessed text. This helps analyze the effect of NLP preprocessing techniques on classification performance. The proposed application provides a simple, lightweight and interpretable approach for demonstrating the complete NLP text-processing pipeline using Python.

2.INTRODUCTION

Natural Language Processing allows computers to work with human language in a meaningful way. Human language is complex because the same word can have different meanings depending on context, and sentences may contain unnecessary words, different word forms, punctuation and grammatical structures.

Before applying machine-learning algorithms to text, the raw input generally needs to be transformed into a suitable representation. NLP preprocessing techniques such as tokenization, stop-word removal, stemming, lemmatization and POS tagging help organize and analyze textual information.

In this project, a complete NLP pipeline is implemented using Python. The system first accepts real-time text from the user. The input is then tokenized, unnecessary stop words are removed, words are stemmed and lemmatized, and grammatical categories are identified using POS tagging.

The processed text is subsequently converted into TF-IDF numerical features. Logistic Regression is used as the classification algorithm to predict whether the input expresses positive or negative sentiment.

The assignment requires the application to demonstrate preprocessing, classification, multiple test inputs and quantitative evaluation using accuracy, precision, recall and F1-score.

3.PROBLEM STATEMENT

Design and implement a Python-based NLP application for real-time text processing using tokenization, stemming/lemmatization, POS tagging, and sentiment or text classification.

The project addresses the following requirements:

1. Explain the role of tokenization, stemming/lemmatization, POS tagging and sentiment classification.
2. Design a suitable workflow for converting raw user input into meaningful linguistic information.
3. Implement tokenization, stop-word removal, stemming, lemmatization and POS tagging.
4. Display the output generated at each preprocessing stage.
5. Extract suitable numerical features using TF-IDF.
6. Apply an appropriate classification algorithm.
7. Predict the sentiment of a new input sentence.
8. Test the application using multiple text inputs.
9. Evaluate the classifier using accuracy, precision, recall and F1-score.
10. Analyze the effect of preprocessing techniques on classification performance.

These requirements correspond directly to the problem statement provided in the assignment document.

4.OBJECTIVES

The main objectives of the project are:

- To develop a Python-based NLP application for real-time text processing.
- To perform tokenization of user-entered sentences.
- To remove unnecessary stop words.
- To apply stemming and lemmatization.
- To identify grammatical categories using POS tagging.
- To convert textual information into numerical features using TF-IDF.
- To develop a sentiment classification model using Logistic Regression.
- To predict whether a new sentence is Positive or Negative.
- To evaluate the classifier using accuracy, precision, recall and F1-score.

5.PROBLEM UNDERSTANDING AND FORMULATION

5.1 Problem Understanding

Raw human language is unstructured and cannot be directly processed effectively by many traditional machine-learning algorithms. Text contains words with different forms, common words that may provide limited information, punctuation and grammatical structures.

Therefore, the problem is formulated as a sequence of text-processing operations that convert raw text into structured linguistic and numerical information.

The application performs the following major operations:

Raw Text → Preprocessing → Feature Extraction → Classification → Evaluation

5.2 Expected Outcome

The expected outcome is a working NLP application that:

- accepts text from a user,
- displays each preprocessing stage,
- identifies POS tags,
- extracts TF-IDF features,
- predicts sentiment,
- supports multiple test inputs,
- and reports classification performance.

5.3 Information and Data Available

The project uses a labelled sentiment dataset containing text sentences and their corresponding sentiment classes.

The classes used in this implementation are:

- Positive
- Negative

The dataset is stored locally in dataset.csv.

5.4 Assumptions

The following assumptions are made:

1. The primary input language is English.
2. The dataset contains representative positive and negative examples.

3. The training and testing data are sufficiently suitable for demonstrating binary sentiment classification.
4. The same preprocessing and TF-IDF transformation used during training are applied to new user input.
5. The user provides meaningful textual input.

5.5 Constraints

The project is designed with the following constraints:

- Python is used as the main programming language.
- NLTK is used for NLP preprocessing.
- Scikit-learn is used for machine learning and evaluation.
- TF-IDF is used for feature extraction.
- Logistic Regression is used for classification.
- The application uses a local dataset.
- No paid external API is required.
- The system should be simple enough for classroom demonstration.

6. REQUIREMENTS AND CONSTRAINTS

6.1 Functional Requirements

The application must:

1. Accept user text.
2. Display the original text.
3. Perform tokenization.
4. Remove stop words.
5. Perform stemming.
6. Perform lemmatization.
7. Perform POS tagging.
8. Train a sentiment classifier.
9. Calculate evaluation metrics.
10. Compare preprocessing approaches.
11. Display results clearly.

6.2 Performance Requirements

The application should:

- provide results quickly for normal-length sentences,
- produce reproducible model results using a fixed random state,
- calculate evaluation metrics correctly,
- avoid unnecessary computational complexity.

6.3 Technical Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM recommended
- Basic processor
- Sufficient storage for Python packages and project files

Software

- Python 3.x
- NLTK
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Antigravity / VS Code or another Python IDE

6.4 Security and Privacy

The application should not unnecessarily store personal information entered by users. If real-world user text is used, it should be handled responsibly and should not be shared without appropriate permission.

7. NLP CONCEPTS USED

7.1 Tokenization

Tokenization is the process of dividing text into smaller units called tokens. A token can be a word, punctuation mark or another meaningful text unit.

For example:

Input:

I really enjoyed this movie!

Tokens:

I, really, enjoyed, this, movie, !

Tokenization is generally one of the first preprocessing operations because subsequent NLP techniques operate on individual tokens.

7.2 Stop-word Removal

Stop words are common words that may contribute less useful information for certain NLP tasks.

Examples include:

- the
- is
- a
- an
- this
- was

For example:

Original:

I really enjoyed this movie

After stop-word removal:

really enjoyed movie

Stop-word removal can reduce unnecessary words and make the text representation more compact.

7.3 Stemming

Stemming reduces words to an approximate root or stem by applying predefined rules.

For example:

playing → play

played → play

studies → studi

The resulting stem may not always be a valid dictionary word.

7.4 Lemmatization

Lemmatization converts a word into its meaningful dictionary base form.

Examples:

running → run

better → good in appropriate linguistic contexts

cars → car

Unlike stemming, lemmatization attempts to produce a valid base word.

7.5 Part-of-Speech Tagging

POS tagging assigns a grammatical category to each word.

Examples:

- NN – Noun
- VB – Verb
- JJ – Adjective
- RB – Adverb
- PRP – Pronoun

For example:

wonderful → JJ

movie → NN

POS tagging provides grammatical information that can be useful for linguistic analysis.

7.6 Logistic Regression

Logistic Regression is a supervised machine-learning classification algorithm.

In this project, it is used for binary sentiment classification:

Positive / Negative

The model learns the relationship between TF-IDF features and sentiment labels from the training dataset.

8. PROPOSED SYSTEM

The proposed system is a modular NLP application that combines linguistic preprocessing and machine learning.

The complete system consists of the following modules:

Module 1 – Input Module

Accepts a sentence from the user.

Module 2 – Preprocessing Module

Performs:

- Tokenization
- Stop-word removal
- Stemming
- Lemmatization
- POS tagging

Module 3 – Feature Extraction Module

Converts processed text into numerical TF-IDF vectors.

Module 4 – Classification Module

Uses Logistic Regression to classify the text.

Module 5 – Evaluation Module

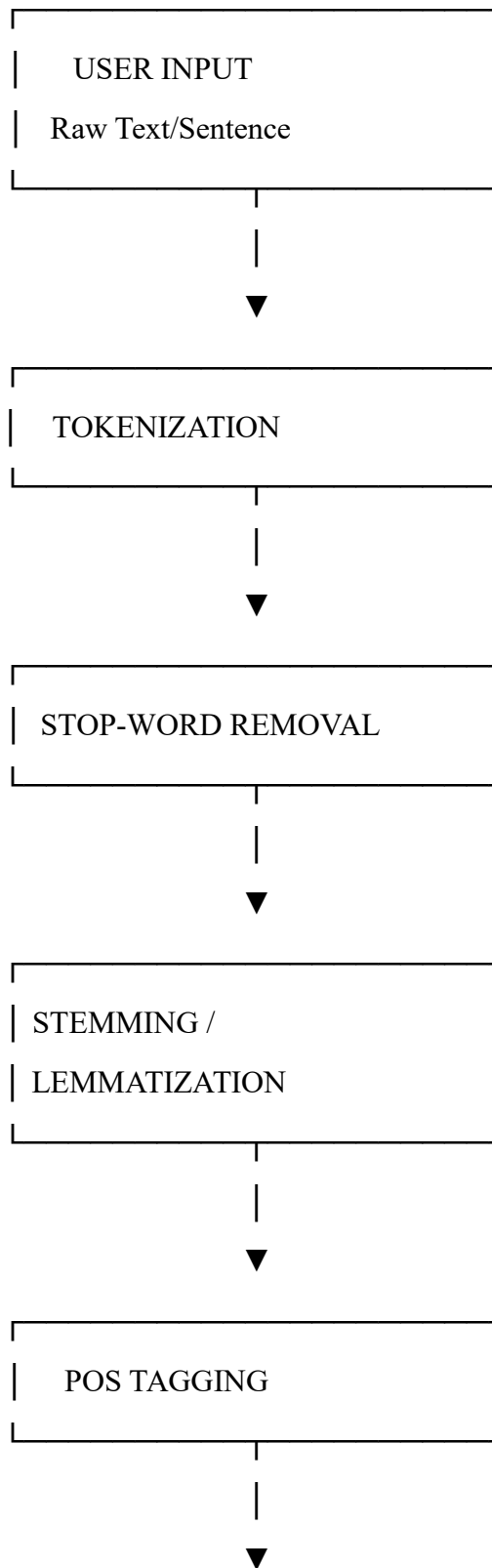
Calculates:

- Accuracy
- Precision
- Recall
- F1-score

Module 6 – Comparison Module

Compares classification performance with and without preprocessing.

SYSTEM ARCHITECTURE



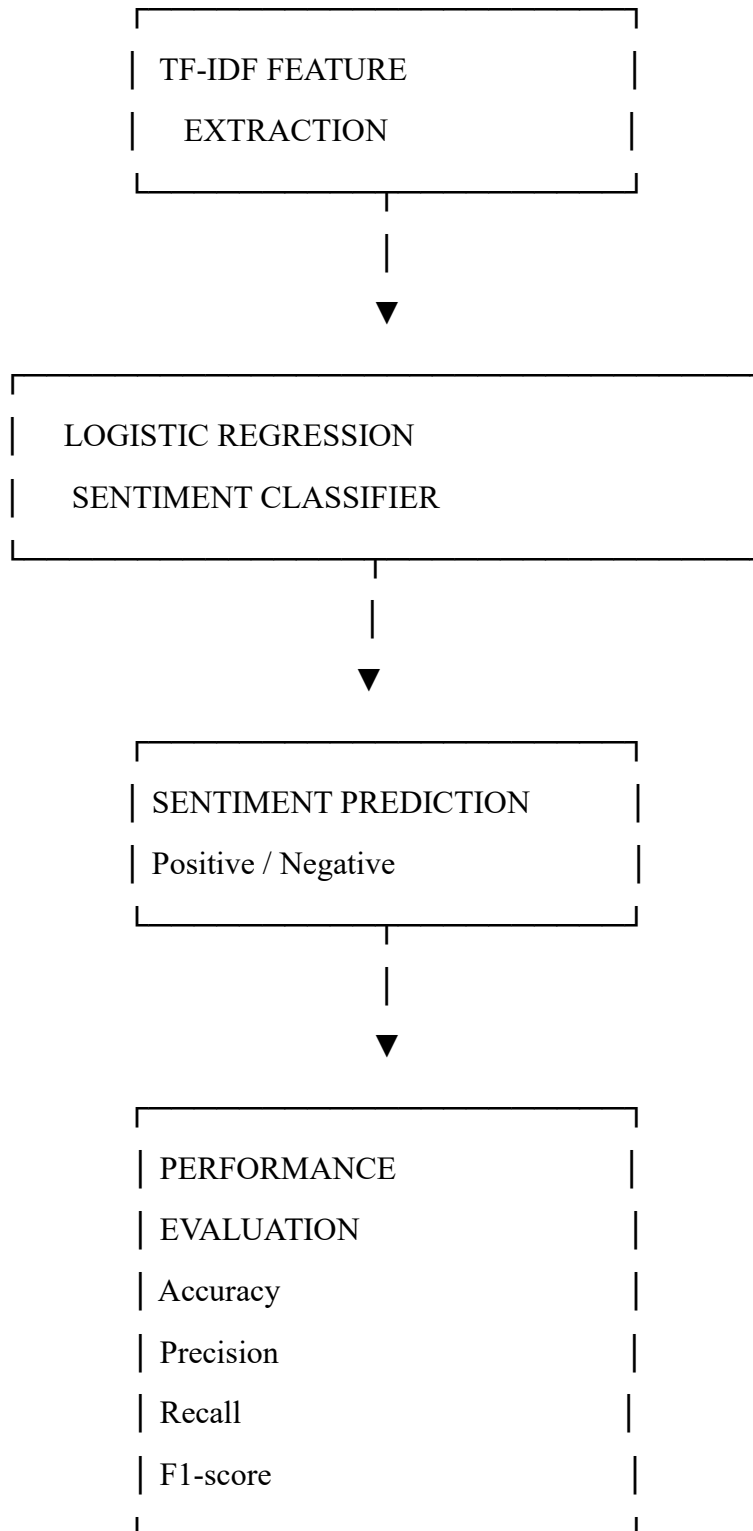


Figure 1: System Architecture of the Proposed NLP Application

10. ALGORITHM AND PSEUDOCODE

10.1 Algorithm

1. Start the application.
2. Load the sentiment dataset.
3. Check the dataset for valid text and labels.
4. Split the dataset into training and testing data.
5. Apply preprocessing to the training and testing text.
6. Tokenize the text.
7. Remove stop words.
8. Apply stemming.
9. Apply lemmatization.
10. Perform POS tagging for analysis.
11. Convert the processed text into TF-IDF features.
12. Train the Logistic Regression classifier.
13. Predict the sentiment labels for the test set.
14. Calculate accuracy.
15. Calculate precision.
16. Calculate recall.
17. Calculate F1-score.
18. Compare raw-text and preprocessed-text performance.
19. Accept new text from the user.
20. Apply the same preprocessing pipeline.
21. Transform the input using the fitted TF-IDF vectorizer.
22. Predict the sentiment.
23. Display the result.
24. Allow the user to test additional sentences.
25. Exit when the user selects the Exit option.

10.2 Pseudocode

START

Load sentiment dataset

Clean dataset

Split dataset into training and testing sets

FOR each training/test sentence:

 Tokenize sentence

 Remove stop words

 Apply stemming

 Apply lemmatization

 Perform POS tagging

END FOR

Convert processed text into TF-IDF vectors

Train Logistic Regression classifier

Predict test-set sentiment

Calculate:

 Accuracy

 Precision

 Recall

 F1-score

Display evaluation results

Compare:

 Raw text performance

 Preprocessed text performance

WHILE user does not select Exit:

 Read user sentence

 Display original sentence

 Tokenize sentence

 Display tokens

 Remove stop words

Display result
 Apply stemming
 Display result
 Apply lemmatization
 Display result
 Perform POS tagging
 Display POS tags
 Transform sentence using fitted TF-IDF vectorizer
 Predict sentiment
 Display Positive / Negative result
 END WHILE
 STOP

11. DATASET DESCRIPTION

The project uses a labelled sentiment dataset containing text sentences and corresponding sentiment labels.

The dataset consists of two major classes:

Sentiment Class	Number of Samples	Example
Positive	5	I really enjoyed this movie.
Negative	5	The service was terrible.
Total	10	

The actual number of samples must be entered after running the implemented project.

The dataset should be reasonably balanced so that one sentiment class does not dominate the training process.

Sample Positive Sentences

1. I really enjoyed this movie.
2. The product is excellent.
3. The service was amazing.
4. I love this experience.
5. This is a wonderful application.

Sample Negative Sentences

1. I hated this movie.
2. The product is terrible.

3. The service was disappointing.
4. I did not enjoy the experience.
5. This application is frustrating.

12. IMPLEMENTATION

12.1 Development Environment

The application is developed using Python.

The major libraries used are:

Library	Purpose
NLTK	NLP preprocessing
Pandas	Dataset handling
NumPy	Numerical operations
Scikit-learn	TF-IDF, Logistic Regression and evaluation
Matplotlib	Performance visualization

12.2 Project Structure

NLP_Assignment_1/

```
|
|— main.py
|— preprocessing.py
|— classifier.py
|— dataset.csv
|— requirements.txt
|— README.md
```

main.py

Controls the overall application and user interaction.

preprocessing.py

Contains NLP preprocessing functions.

classifier.py

Contains:

- dataset loading,
- TF-IDF vectorization,
- Logistic Regression,

- prediction,
- evaluation metrics.

dataset.csv

Contains the labelled sentiment data.

requirements.txt

Contains the Python packages required by the application.

README.md

Contains project information and execution instructions.

12.3 Installation

The required libraries can be installed using:

pip install nltk pandas numpy scikit-learn matplotlib

If required on Windows:

python -m pip install nltk pandas numpy scikit-learn matplotlib

The required NLTK resources are downloaded by the application or installed during setup.

13. PREPROCESSING RESULTS

13.1 Original Text

For demonstration, the following sentence can be entered:

Input:

I really enjoyed this wonderful movie!

The application first displays the complete original input before preprocessing.

13.2 Tokenization

Tokenization divides the input sentence into individual tokens.

Example:

Input:

I really enjoyed this wonderful movie!

Tokens:

I

really

enjoyed

this

wonderful

movie

!

13.3 Stop-word Removal

The stop-word removal stage removes selected common words.

Example:

Before:

I really enjoyed this wonderful movie

After:

really enjoyed wonderful movie

13.4 Stemming

Stemming reduces words to approximate root forms.

Example:

enjoyed → enjoy

wonderful → wonder

movie → movi

13.5 Lemmatization

Lemmatization converts words to meaningful base forms.

Example:

enjoyed → enjoy

movies → movie

13.6 POS Tagging

POS tagging assigns grammatical categories to the tokens.

Example:

really → RB

enjoyed → VBD

wonderful → JJ

movie → NN

14. SENTIMENT CLASSIFICATION

14.1 Feature Extraction Using TF-IDF

After preprocessing, the text is converted into numerical features using TF-IDF.

The TF-IDF representation allows the machine-learning model to work with numerical vectors instead of raw text.

The vectorizer is fitted using the training data and then used to transform both testing data and new user input.

This approach helps avoid data leakage because the vocabulary and IDF information are learned from the training data.

14.2 Logistic Regression Classification

Logistic Regression is selected as the classification algorithm because it is suitable for binary classification and works effectively with sparse text representations such as TF-IDF vectors.

The classifier learns from the training examples and associates text features with the corresponding sentiment labels.

The two output classes are:

POSITIVE

NEGATIVE

14.3 Sample Sentiment Prediction

Input:

I really enjoyed this wonderful movie!

Processing:

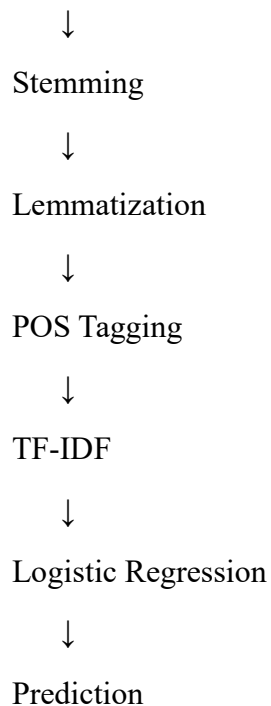
Original Text

↓

Tokenization

↓

Stop-word Removal



Output:

Predicted Sentiment: POSITIVE

Confidence: _____

15. MULTIPLE INPUT TESTING

The application should be tested with multiple sentences to verify its behaviour.

Test No.	Input Sentence	Actual/Expected Sentiment	Predicted Sentiment
1	I really enjoyed this movie.	Positive	_____
2	The service was terrible.	Negative	_____
3	This product is excellent.	Positive	_____
4	I am disappointed with this experience.	Negative	_____
5	The application is very useful.	Positive	_____
6	The product quality is poor.	Negative	_____

The predicted values must be obtained by running the actual classifier.

16. PERFORMANCE EVALUATION

The performance of the classifier is evaluated using four major metrics.

16.1 Accuracy

Accuracy measures the proportion of correctly classified instances among all test instances.

Accuracy = Correct Predictions / Total Predictions

16.2 Precision

Precision measures how many instances predicted as a particular class were actually members of that class.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

16.3 Recall

Recall measures how many actual instances of a class were correctly identified.

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

16.4 F1-score

F1-score is the harmonic mean of precision and recall.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

16.5 Model Performance Table

Enter the actual results generated by your program.

Metric	Without Preprocessing	With Preprocessing
Accuracy	XX.XX%	XX.XX%
Precision	XX.XX%	XX.XX%
Recall	XX.XX%	XX.XX%
F1-score	XX.XX%	XX.XX%

17. PREPROCESSING COMPARISON

An important part of the project is comparing the classification performance before and after preprocessing.

Experiment 1 – Without Preprocessing

Raw Text



TF-IDF



Logistic Regression



Prediction

Experiment 2 – With Preprocessing

Raw Text



Tokenization



Stop-word Removal



Stemming / Lemmatization



TF-IDF



Logistic Regression



Prediction

Discussion

The comparison should be based on the actual experimental results.

If preprocessing improves the results, it may indicate that removing unnecessary words and normalizing word forms helped the classifier focus on useful textual information.

If preprocessing reduces the results, this should also be discussed. In some datasets, removing stop words or applying stemming may remove information that is useful for sentiment classification. For example, negation words can sometimes be important in determining sentiment.

Therefore, preprocessing should not automatically be assumed to improve every classification task. Its effect depends on the dataset, preprocessing strategy and classification objective.

18. RESULTS AND SCREENSHOTS

18.1 Application Main Menu

The application provides a menu for interacting with the NLP system.

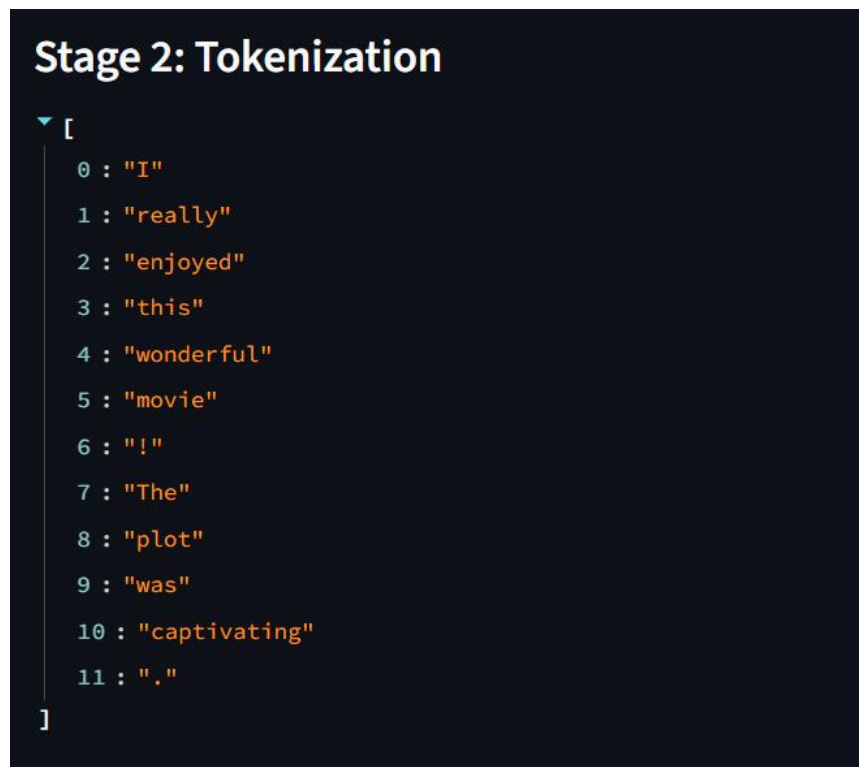
Example:

REAL-TIME NLP TEXT ANALYZER

1. Analyze Text
2. Test Multiple Sentences
3. Show Model Performance
4. Compare Preprocessing Performance
5. Exit

18.2 Tokenization Result

The tokenization stage should display the tokens extracted from the input sentence.



```
Stage 2: Tokenization
[
  0 : "I"
  1 : "really"
  2 : "enjoyed"
  3 : "this"
  4 : "wonderful"
  5 : "movie"
  6 : "!"
  7 : "The"
  8 : "plot"
  9 : "was"
  10 : "captivating"
  11 : "."
]
```

18.3 Stop-word Removal Result

Stage 3: Stop-word Removal

['really', 'enjoyed', 'wonderful', 'movie', 'plot', 'captivating']

18.4 Stemming Result

Stage 4: Stemming (PorterStemmer)

['realli', 'enjoy', 'wonder', 'movi', 'plot', 'captiv']

18.5 Lemmatization Result

Stage 5: Lemmatization (WordNetLemmatizer)

['really', 'enjoyed', 'wonderful', 'movie', 'plot', 'captivating']

18.6 POS Tagging Result

Stage 6: POS Tagging (Part-of-Speech)

	Token	POS Tag
0	really	RB
1	enjoyed	VRN
2	wonderful	JJ
3	movie	NN
4	plot	NN
5	captivating	VBG

18.7 Sentiment Prediction

Enter a sentence to analyze through the NLP Pipeline:

User Input Text:

I really enjoyed this wonderful movie! The plot was captivating.

Analyze & Classify

Sentiment Prediction Result

 **POSITIVE SENTIMENT**

Confidence Score: 62.31%

18.8 Multiple Inputs

Input Text	Preprocessed Form	Predicted Sentiment	Confidence
I absolutely love this product, it works beyond my expectations!	absolutely love product work beyond expectation	POSITIVE	54.03%
This service was terrible, slow, and completely unresponsive.	service terrible slow completely unresponsive	NEGATIVE	67.44%
The movie was amazing with breathtaking visual effects and great acting.	movie amazing breathtaking visual effect great acting	POSITIVE	57.49%
I am extremely disappointed with the experience and demand a refund.	extremely disappointed experience demand refund	NEGATIVE	53.51%

18.9 Model Performance

Main Classifier Performance (Experiment 2: Preprocessed Text)

Accuracy	Precision	Recall	F1-Score
60.00%	56.25%	90.00%	69.23%

18.10 Classification Report

Detailed Classification Report

precision recall f1-score support

Negative 0.75 0.30 0.43 10

Positive 0.56 0.90 0.69 10

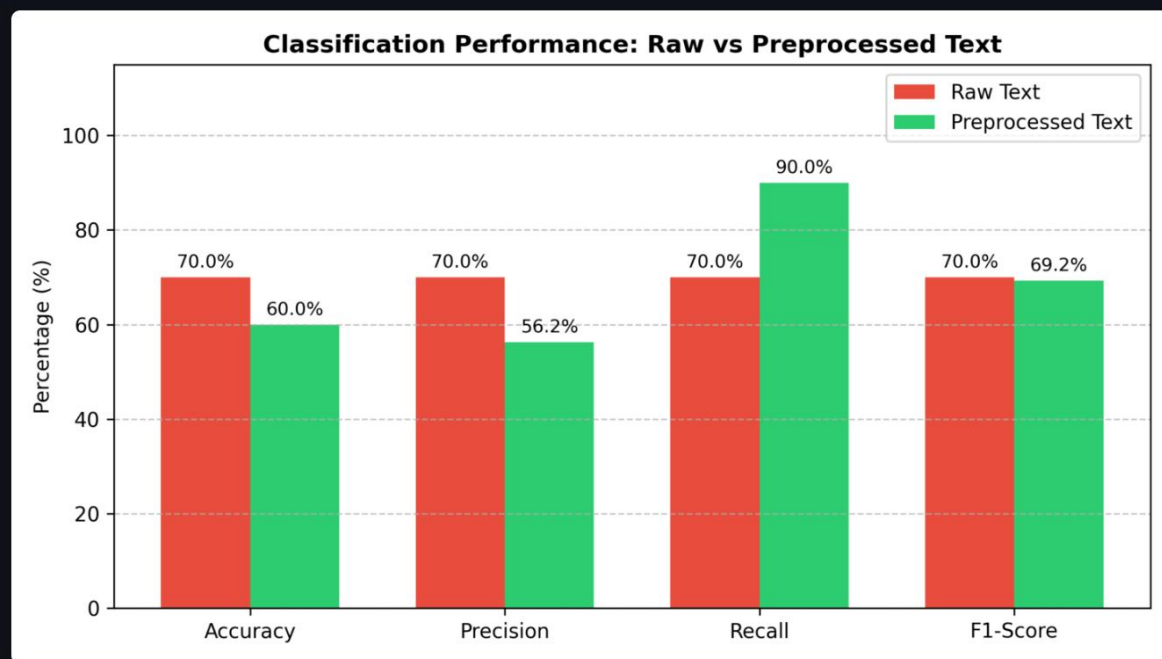
accuracy 0.60 20

macro avg 0.66 0.60 0.56 20

weighted avg 0.66 0.60 0.56 20

18.11 Performance Chart

Visual Comparison Chart



19. ANALYSIS AND ENGINEERING DECISION

The proposed system demonstrates the complete transformation of raw text into a machine-learning classification result.

Tokenization provides the basic units required for further text processing. Stop-word removal can reduce frequently occurring words that may not contribute significantly to the selected task. Stemming and lemmatization reduce variation among word forms. POS tagging provides additional linguistic information about the grammatical role of words.

TF-IDF was selected for feature extraction because it converts textual data into numerical vectors while assigning greater importance to terms that are informative within the dataset.

Logistic Regression was selected as the classification algorithm because the problem is a binary classification problem and Logistic Regression is relatively simple, efficient and suitable for TF-IDF-based text features.

The final decision between the raw-text and preprocessed-text approaches should be based on the actual evaluation metrics generated during experimentation.

20. LIMITATIONS

The developed NLP application has several limitations.

20.1 Dataset Size

The quality of classification depends on the size and diversity of the dataset. A small dataset may not represent all possible language patterns.

20.2 Vocabulary Limitation

The classifier may perform poorly when it receives words or expressions that are not adequately represented in the training data.

20.3 Context Understanding

TF-IDF represents word importance but does not provide deep contextual understanding of sentences.

20.4 Sarcasm

Sarcastic statements can be difficult to classify correctly because their literal words may express a different sentiment from their intended meaning.

20.5 Negation

Expressions such as:

not good

or

not very useful

may require careful handling because the presence of the word "not" can significantly change the meaning.

20.6 Domain Dependence

A model trained using general reviews may not perform equally well on technical, legal, medical or social-media text.

20.7 Classical Model Limitation

Logistic Regression with TF-IDF is lightweight and interpretable, but it does not capture contextual relationships as deeply as modern transformer-based language models.

21. BROADER CONSIDERATIONS

21.1 Ethical Considerations

Sentiment classification should not be treated as a completely reliable judgment of a person's feelings or opinions. Incorrect predictions may occur, particularly with ambiguous, sarcastic or context-dependent language.

21.2 Privacy

If the application is extended to real-world user data, text should be processed responsibly. Personal or sensitive information should not be unnecessarily stored.

21.3 Bias

If the training dataset contains biased, unbalanced or unrepresentative language, the resulting classifier may reproduce those biases.

21.4 Accessibility

The simple text-based interface makes the application easy to operate and demonstrate.

21.5 Sustainability

The selected TF-IDF and Logistic Regression approach requires comparatively modest computational resources, making it suitable for a lightweight educational application.

21.6 Professional Responsibility

NLP systems should be evaluated before being used in real-world decision-making. Model performance metrics, limitations and possible sources of error should be communicated clearly.

22. APPLICATIONS

The developed NLP approach can be extended to several practical applications.

22.1 Customer Feedback Analysis

Companies can analyze customer reviews and identify overall positive or negative feedback.

22.2 Product Review Analysis

Online product reviews can be automatically categorized based on sentiment.

22.3 Social Media Monitoring

Publicly available social-media text can be analyzed to identify general sentiment trends.

22.4 Opinion Mining

The system can be extended to identify opinions about products, services or topics.

22.5 Chatbot Analysis

Sentiment analysis can be incorporated into conversational systems to identify the emotional polarity of user messages.

23. CONCLUSION

This project successfully demonstrates a complete Python-based NLP pipeline for real-time text processing and sentiment classification. The application accepts raw user text and performs tokenization, stop-word removal, stemming, lemmatization and POS tagging. These preprocessing techniques transform unstructured text into a more suitable form for further analysis. The processed text is represented using TF-IDF features and classified using Logistic Regression. The application supports sentiment prediction for new user inputs and allows multiple sentences to be tested.

The model is evaluated using accuracy, precision, recall and F1-score. A comparison between raw-text and preprocessed-text classification is also performed to understand the effect of preprocessing.

The project provides practical experience in applying NLP concepts, machine-learning techniques, Python programming and model evaluation to a real-world text classification problem. The final performance conclusions should be based on the actual experimental results obtained from the implemented application.

25. FUTURE ENHANCEMENTS

The following enhancements can be considered:

1. Increase the size of the training dataset.
2. Use real-world sentiment datasets.
3. Compare Logistic Regression with Naive Bayes and SVM.
4. Implement transformer-based models such as BERT.
5. Add multilingual NLP support.
6. Add emotion classification.
7. Improve negation handling.
8. Improve sarcasm detection.
9. Develop a Streamlit web interface.
10. Provide graphical dashboards for sentiment analysis.
11. Store historical evaluation results for analysis.
12. Extend the system to domain-specific datasets.

26. REFERENCES

The following references may be included, provided they are actually used during implementation:

1. Jurafsky, D., and Martin, J. H., *Speech and Language Processing*, Pearson.
2. Bird, S., Klein, E., and Loper, E., *Natural Language Processing with Python*, O'Reilly Media.
3. NLTK Documentation, Natural Language Toolkit.
4. Scikit-learn Documentation, Machine Learning in Python.
5. Pandas Documentation.
6. NumPy Documentation.
7. Matplotlib Documentation.
8. The dataset source used for the sentiment classification experiment.